

README

Part 5: WaveNet — 从展平到层次化融合

> 从"一口气看完所有上下文"到"从局部到全局的层次化融合"！

章节导航

序号	章节	内容
01	[PyTorch 化：让代码更优雅](01_pytorchify.md)	Embedding/Flatten/Sequential 模块化、BatchNorm 的训练/推理坑
02	[WaveNet 架构：层次化融合](02_wavenet_architecture.md)	为什么要层次化、FlattenConsecutive、Linear 支持多维输入
03	[训练与 Bug 修复](03_training_and_bugs.md)	BatchNorm 3D bug、放大训练、卷积预览

学习路线图

Part 3 (BatchNorm)

| "网络稳定了，但性能到瓶颈了..."

	Part 5: WaveNet	
	① PyTorch 化代码 — 模块化重构	———> 01_pytorchify.md
	② WaveNet 架构 — 层次化融合	———> 02_wavenet_architecture.md
	③ 训练修复 — 3D BatchNorm bug	———> 03_training_and_bugs.md

| "理解了 WaveNet，接下来是让 token 自己决定看谁..."

Part 6: Transformer/GPT ([开始学习](../Part6_transformer/tutorial/README.md))

学完这一部分你能...

- 把散乱的管理参数重构成 PyTorch 风格的模块化代码
- 理解 Sequential 容器和 `parameters()` 统一接口
- 掌握 FlattenConsecutive 层：逐步融合上下文
- 理解 Linear 层天然支持多维输入的洞察
- 修复 BatchNorm1D 的 3D 输入 bug
- 构建 WaveNet 层次化架构，验证 loss 降到 < 2.0

课后作业

完成教程后，去这里做练习：

[Assignment 5](../../assignments/assignment_5/)

相关资源

- Andrej Karpathy 原视频：[Building makemore Part 5: Building a WaveNet](https://www.youtube.com/watch?v=t3YJ5hKiMQ0)
- Van den Oord et al. 2016 论文：[WaveNet: A Generative Model for Raw Audio](https://arxiv.org/abs/1609.03499)
- Dilated Causal Convolutions：WaveNet 的卷积等价形式

[← 上一章：Part 4 Backpropagation](../../Part4_backprop/tutorial/README.md)

01_pytorchify

01 — PyTorch 化：让代码更优雅

> 从字典管理层到模块化层对象，让代码的形状匹配思想的形状。

从 Part 3 的问题出发

Part 3 的深层网络用字典管理层：

```
`python
layers = []
layers.append({'type': 'linear', 'W': W, 'b': b})
layers.append({'type': 'batchnorm', 'bn': bn})
```

forward 时要判断类型

```
for layer in layers:
    if layer['type'] == 'linear':
        x = x @ layer['W'] + layer['b']
    elif layer['type'] == 'batchnorm':
        x = layer['bn'](x)
```

这行得通，但很丑。forward 逻辑被 if/elif 污染，每加一种层就要改 forward 函数。

更好的方式：每个层是一个对象，有统一的 `__call__` 接口。

PyTorch 化的层

Embedding 层

```
`python
class Embedding:
def __init__(self, num_embeddings, embedding_dim):
self.weight = torch.randn((num_embeddings, embedding_dim))

def __call__(self, IX):
self.out = self.weight[IX]
return self.out

def parameters(self):
return [self.weight]
```

封装了 `C[IX]` 操作。之前写 `emb = C[X]`，现在写 `emb = embedding(X)`。

Flatten 层

```
`python
class Flatten:
def __call__(self, x):
self.out = x.view(x.shape[0], -1)
return self.out

def parameters(self):
return []
```

封装了 `view(0, -1)` 操作。

Linear 层

```
`python
class Linear:
def __init__(self, fan_in, fan_out, bias=True):
self.weight = torch.randn((fan_in, fan_out)) / fan_in ** 0.5
self.bias = torch.zeros(fan_out) if bias else None

def __call__(self, x):
self.out = x @ self.weight
if self.bias is not None:
self.out += self.bias
return self.out

def parameters(self):
return [self.weight] + ([self.bias] if self.bias is not None else [])
```

自带 Kaiming 初始化。

Sequential 容器

```
`python
class Sequential:
def __init__(self, layers):
self.layers = layers

def __call__(self, x):
for layer in self.layers:
x = layer(x)
self.out = x
return self.out

def parameters(self):
return [p for layer in self.layers for p in layer.parameters()]
`
```

一行 `model.parameters()` 收集所有参数！

用 Sequential 重构网络

```
`python
model = Sequential([
Embedding(vocab_size, n_embd),
Flatten(),
Linear(n_embd * block_size, n_hidden, bias=False),
BatchNorm1d(n_hidden),
Tanh(),
Linear(n_hidden, vocab_size),
])
```

所有参数一行搞定

```
for p in model.parameters():
p.requires_grad = True
`
```

对比 Part 3 的写法，清爽多了。

BatchNorm 的训练/推理坑

BatchNorm 有两种模式，通过 `self.training` 标志切换：

模式	统计量来源	更新 running stats ?
Training ('training=True')	当前 mini-batch	是
Eval ('training=False')	running statistics	否

忘记切换的后果：

- 训练时用 eval 模式 → running stats 不更新 → 推理时统计量不准
- 推理时用 training 模式 → 输出取决于当前 batch → 不确定性

```
`python
训练
```

```
for layer in model.layers:
    if hasattr(layer, 'training'):
        layer.training = True
```

评估

```
for layer in model.layers:
    if hasattr(layer, 'training'):
        layer.training = False
```

> PyTorch 的 `nn.Module` 用 `model.train()` / `model.eval()` 做这件事。我们手动管理是为了理解底层机制。

代码参考

[01_pytorchify_layers.py](../scripts/01_pytorchify_layers.py) — 完整的 PyTorch 化重构

下一步

代码模块化了，接下来要做的是改变网络结构——从"展平所有上下文"到"层次化融合"。

[02 — WaveNet 架构](02_wavenet_architecture.md)

02_wavenet_architecture

02 — WaveNet 架构：层次化融合

> 从"一口吞下 8 个字符"到"先消化 2 个，再消化 4 个，最后消化 8 个"。

动机：为什么不能直接展平？

之前的方法：8 个字符的 embedding 展平成 80 维向量，直接送进 Linear 层。

8 chars → 展平成 80 维 → Linear(80, 200) → ...

问题：80 维向量一次性混合所有信息，网络很难学到"相邻字符之间的关系"。

WaveNet 的思路：逐步融合，从局部到全局。

8 chars → 4 bigrams → 2 fourgrams → 1 eightgram

每一步只融合相邻的两个向量，形成层次化的特征提取。

树状融合结构

```

\
[8-gram 表征]
/\
[4-gram] [4-gram]
/\ /\
[bigram] [bigram] [bigram] [bigram]
/\ /\ /\ /\
c1 c2 c3 c4 c5 c6 c7 c8
\

```

从底向上，每层把两个相邻向量融合成一个。

关键洞察：Linear 支持多维输入

```

\python
x = torch.randn(4, 8, 10) # (B, T, C)
W = torch.randn(10, 20) # (C, H)
y = x @ W # (4, 8, 20) ← 自动广播！
\

```

Linear 层只在最后一个维度做矩阵乘法。前面的维度 (B, T) 自动保留。

这意味着我们可以：

1. 不展平 T 维度
2. 直接在 3D tensor 上做 Linear 变换
3. 在每一步 FlattenConsecutive 后接 Linear

FlattenConsecutive 层

```

\python
class FlattenConsecutive:
    def __init__(self, n):
        self.n = n

    def __call__(self, x):
        B, T, C = x.shape
        (B, T, C) → (B, T//n, C*n)

        x = x.view(B, T // self.n, C * self.n)
        self.out = x
        return self.out
\

```

示例：

```

\
输入: (B, 8, 10) — 8 个字符，每个 10 维
FC(2): (B, 4, 20) — 4 个 bigram，每个 20 维
FC(2): (B, 2, 40) — 2 个 fourgram，每个 40 维
FC(2): (B, 1, 80) — 1 个 eightgram，80 维
\

```

WaveNet 完整架构

```
`python
model = Sequential([
    Embedding(vocab_size, n_embd),
```

层 1: $8 \rightarrow 4$

```
    FlattenConsecutive(2),
    Linear(n_embd * 2, n_hidden, bias=False),
    BatchNorm1d(n_hidden),
    Tanh(),
```

层 2: $4 \rightarrow 2$

```
    FlattenConsecutive(2),
    Linear(n_hidden * 2, n_hidden, bias=False),
    BatchNorm1d(n_hidden),
    Tanh(),
```

层 3: $2 \rightarrow 1$

```
    FlattenConsecutive(2),
    Linear(n_hidden * 2, n_hidden, bias=False),
    BatchNorm1d(n_hidden),
    Tanh(),
```

输出层

```
    Linear(n_hidden, vocab_size),
])
```

扩大上下文窗口

block_size 从 3 增大到 8：

block_size	上下文示例	验证 loss
3	`...e`mma $\rightarrow$ `m`	~2.10
8	`....emma` $\rightarrow$ `n`	~2.02

仅靠更多上下文就能提升性能。但直接展平 8 个字符不如层次化融合效果好。

view vs cat

```
`python
```

方法 1: view（高效，无内存拷贝）

```
x = x.view(B, T // 2, C * 2)
```

方法 2: cat (低效, 有内存拷贝)

```
left = x[:, ::2, :]  
right = x[:, 1::2, :]  
x = torch.cat([left, right], dim=2)
```

view 只是改变了 tensor 的视图 (stride 和 shape), 零拷贝。cat 会分配新内存。在训练循环中, 这个差异会累积。

代码参考

- [03_increase_context.py](../scripts/03_increase_context.py) — block_size 扩大到 8
- [04_flatten_consecutive.py](../scripts/04_flatten_consecutive.py) — FlattenConsecutive 演示
- [05_wavenet_architecture.py](../scripts/05_wavenet_architecture.py) — 完整 WaveNet 架构

下一步

架构搭好了, 但训练时发现 BatchNorm 在 3D 输入上有 bug。

[03 — 训练与 Bug 修复](03_training_and_bugs.md)

03_training_and_bugs

03 — 训练与 Bug 修复

> 波折之后才能到达终点。BatchNorm 的 3D bug 是 WaveNet 训练的关键障碍。

BatchNorm1D 的 3D Bug

问题

WaveNet 中间层的输出是 3D tensor (**B, T, C**), 但我们之前写的 BatchNorm1d 只考虑了 2D:

```
`python
```

Bug 版本

```
xmean = x.mean(dim=0, keepdim=True) # 对 3D 输入: shape (1, T, C)
```

这会导致每个时间步独立归一化, 而不是在整个 batch $\times$ time 上归一化。

修复

```
`python
```



```

def __call__(self, x):
    if self.training:
        if x.ndim == 2:
            dim_reduce = 0 # (B, C) → 在 B 维度上 reduce
        else:
            dim_reduce = (0, 1) # (B, T, C) → 在 B 和 T 维度上 reduce

    xmean = x.mean(dim=dim_reduce, keepdim=True) # shape (1, 1, C) 或 (1, C)
    xvar = x.var(dim=dim_reduce, keepdim=True)

```

关键：

- `running_mean` 和 `running_var` 始终是 1D tensor (C,)
- 2D eval：`unsqueeze(0)` → (1, C) 广播到 (B, C)
- 3D eval：`unsqueeze(0).unsqueeze(0)` → (1, 1, C) 广播到 (B, T, C)

验证

```

python
3D 输入 (B=32, T=4, C=10)

x3d = torch.randn(32, 4, 10)
bn = BatchNorm1d(10)
y = bn(x3d)

print(f"running_mean shape: {bn.running_mean.shape}") # 应该是 (10,)
print(f"输出均值: {y.mean():.6f}") # 应该接近 0
print(f"输出标准差: {y.std():.6f}") # 应该接近 1

```

放大训练

把网络容量增大：

参数	小模型	放大模型
n_embd	10	24
n_hidden	68	128
参数量	~22K	~170K
batch_size	32	128
训练步数	20K	50K

```

python
model = Sequential([
    Embedding(vocab_size, 24), # 更大的 embedding
    FlattenConsecutive(2), Linear(48, 128, bias=False), BatchNorm1d(128), Tanh(),
    FlattenConsecutive(2), Linear(256, 128, bias=False), BatchNorm1d(128), Tanh(),
    FlattenConsecutive(2), Linear(256, 128, bias=False), BatchNorm1d(128), Tanh(),
    Linear(128, vocab_size),
])

```

性能对比

模型	block_size	验证 loss
MLP (Part 2)	3	~2.10
深层 BN (Part 3)	3	~2.07
WaveNet 小模型	8	~2.07
WaveNet 放大	**8**	**~1.99**

验证 loss 首次降到 2.0 以下！

卷积预览：WaveNet 的另一种视角

我们用 FlattenConsecutive + Linear 实现的层次融合，本质上等价于 Dilated Causal Convolution（膨胀因果卷积）。

传统卷积（kernel=2）：

c1 c2 c3 c4 c5 c6 c7 c8

|—| |—| |—| |—| |—| |—| |—|

(每次看 2 个相邻字符)

膨胀卷积（dilation=2）：

c1 c2 c3 c4 c5 c6 c7 c8

|—————| |—————| |—————|

(每次看间隔 2 的字符)

膨胀卷积（dilation=4）：

c1 c2 c3 c4 c5 c6 c7 c8

|—————|

(每次看间隔 4 的字符)

这种卷积的感受野指数增长，和我们的层次融合完全等价。区别只是实现方式：

- 我们的方式：FlattenConsecutive + Linear（更直观）
- 卷积方式：Dilated Causal Conv（更高效，特别是对于长序列）

> 这就是为什么论文叫 "WaveNet"——它最初是为音频波形设计的，用膨胀因果卷积处理超长序列。

代码参考

- [06_batchnorm_3d_fix.py](../scripts/06_batchnorm_3d_fix.py) — BatchNorm 3D bug 修复
- [07_scaled_wavenet.py](../scripts/07_scaled_wavenet.py) — 放大训练到 loss < 2.0

课后作业

动手实践！去完成练习：

[Assignment 5](../../assignments/assignment_5/)